

Manual Prático de Defesa (PDD)

Projeto: Automação do Processo de Relatórios Administrativos — Equipe 04

Este documento foi **reorganizado 100% por pessoa**. Assim, cada integrante pode ir diretamente para a sua seção e saber exatamente o que deve explicar, como demonstrar na prática e quais arquivos abrir.

☐ PESSOA 1 — Automação / RPA

Responsabilidade Principal: GERP fake, login, download, integração Playwright, Circuit Breaker e Fallback.

O que você vai defender na banca:

- 1. A Visão Geral do Fluxo (Perguntas 1 e 11):** Você explicará que o robô orquestra um pipeline de 6 etapas: Extração -> Tratamento -> Cruzamento -> Análise -> Identificação -> Relatório. Essa modularização facilita a manutenção do código.
- 2. Tratamento de Erros e Contingência (Perguntas 4 e 5):** O sistema tem três camadas de erro:
 - Local: Tratando timeouts do Playwright.
 - Contingência: Se falhar a extração web, o robô não quebra; ele usa um CSV de fallback guardado localmente.
 - Global: Encerramento com erro no orquestrador principal para avisar o SO.
- 3. Circuit Breaker (Pergunta 6):** Para não bombardear o GERP fora do ar, o sistema fecha o circuito, aciona o Fallback, espera 60 segundos (Half-Open) e só depois tenta conectar de novo.

O que mostrar na prática (Arquivos):

- **src/main.py:** Aponte para a função `run()` onde as 6 etapas são chamadas.
 - **src/circuit_breaker.py:** Mostre as regras de estado (Fechado, Aberto, Meio-Aberto).
 - **logs/circuit_breaker_state.json:** Se quiser demonstrar, rode o script, derrube o servidor mock, rode 3x até falhar e mostre o arquivo mudando para `"estado": "ABERTO"`.
-

||| PESSOA 2 — Dados / Regras de Negócio

Responsabilidade Principal: Leitura CSV/XLSX, tratamento, cruzamento, desvios matemáticos.

O que você vai defender na banca:

- 1. A Regra do Cruzamento (Pergunta 2):** Você explicará que o sistema une as planilhas

usando um `Outer Join` (do Pandas). Isso é fundamental porque, se um projeto existe na base financeira mas sumiu da base de produção (ou vice-versa), o robô não vai simplesmente ignorá-lo.

2. Cálculos e Regra Principal (Perguntas 2 e 3):

- **Desvio Financeiro:** `Custo Realizado - Faturamento Previsto`
- **Desvio de Produção:** `Unidades Planejadas - Unidades Produzidas`
- Você explicará os 3 níveis (NORMAL, ATENÇÃO, CRÍTICO).
- **Caso PROJ_REF_02:** Explicar que ele é classificado como CRÍTICO porque estourou o limite financeiro E apresentou problemas físicos na produção (atraso). É um desastre duplo.

O que mostrar na prática (Arquivos):

- **src/data_processor.py:** Vá na função `cruzar_dados` e mostre o `pd.merge(..., how='outer')`.
- Ainda em **src/data_processor.py:** Role até a função `_classificar` e mostre os `ifs` que geram a classificação de risco (Normal, Atenção, Crítico).

PESSOA 3 — Análise / Relatório / Alertas

Responsabilidade Principal: Divergências, criticidade, relatório, logs, alertas e fuso horário.

O que você vai defender na banca:

1. Separação de Logs e Eventos (Pergunta 7): Você vai destacar que o robô não joga tudo numa tela só. Temos logs normais da execução (para debug) e um arquivo exclusivo chamado `alerts.log` que SÓ recebe as catástrofes (projetos críticos ou falhas irre recuperáveis).

2. Preocupação com Fuso Horário (Pergunta 7): Tudo no sistema (desde o nome do relatório até os logs) usa `TZ=America/Manaus`. Você vai defender que isso evita o clássico problema de rodar o robô num servidor americano e as datas do relatório saírem em UTC.

3. Geração do Relatório: O produto final é um relatório executivo legível (txt) e um banco rastreável JSON para auditoria futura.

O que mostrar na prática (Arquivos):

- Pasta **logs/**: Abra o `alerts.log` e mostre o alerta crítico sendo gerado isoladamente lá.
- Pasta **output/**: Mostre como o `relatorio_final_diretoria.txt` ficou preenchido com as tags substituídas pelos valores do robô.
- **src/logger.py:** Mostre a linha do código onde a `ZoneInfo` força o fuso de Manaus.

PESSOA 4 — Qualidade / DevOps / Integração

Responsabilidade Principal: Pytest, Docker, Compose, `.env`, execução da aplicação inteira.

O que você vai defender na banca:

1. Orquestração Docker (Pergunta 9): O Docker Compose sobe 2 serviços conectados: o mock do GERP e o robô em si. O robô tem um "Depends On" configurado para só dar o bote quando o GERP estiver rodando (evitando falhas). O robô usa usuário "não-root" por segurança.

2. Automação de Qualidade (Perguntas 8 e 10): O projeto roda 18 testes automatizados em Pytest garantindo as regras de negócio. Qualquer *push* feito pro GitHub aciona a esteira do Github Actions (CI/CD) que roda esses testes sozinho.

O que mostrar na prática (Arquivos):

1. No Dockerfile (Foco em Segurança):

- **Onde mostrar:** Abra o Dockerfile, mostre as linhas 10 (`useradd...`) e 21 (`USER robo`).
- **A Conclusão (O que falar):** *"A maioria dos containers Docker roda por padrão como 'root', o que é uma falha grave de segurança. Para seguir as práticas de DevSecOps, nós criamos um usuário restrito chamado 'robo'. Isso significa que o nosso container é cego para o sistema operacional hospedeiro, garantindo total segurança na implantação."*

2. No docker-compose.yml (Foco em Estabilidade):

- **Onde mostrar:** Abra o `docker-compose.yml`. Destaque as linhas do `healthcheck` (linha 15) e a trava `depends_on: condition: service_healthy` (linha 28).
- **A Conclusão (O que falar):** *"O maior problema em microserviços é o tempo de inicialização. Se o robô iniciar milissegundos antes do servidor mock, ele quebra. Nós resolvemos isso criando um 'healthcheck'. Nosso robô fica aguardando inteligentemente e só dá o primeiro passo quando o Docker confirma que o sistema GERP está 'healthy' (totalmente no ar). Isso garante zero quedas por dessincronização."*

3. No Terminal da IDE (Foco em Qualidade):

- **Onde mostrar:** Abra o terminal e rode `pytest -v` ao vivo. O professor verá a tela inteira de testes passando verde.
- **A Conclusão (O que falar):** *"Para fechar nosso ciclo DevOps, nós não confiamos apenas em testes manuais. Temos 18 testes automatizados (dentro da pasta `tests/`) garantindo as regras matemáticas e o comportamento do robô."*

4. No Arquivo ci.yml (Foco em Integração Contínua - CI/CD):

- **Onde mostrar:** Abra a pasta `.github/workflows/` e clique no arquivo `ci.yml`. Aponte para a linha 29 (`run: pytest tests/ -v`).
- **A Conclusão (O que falar):** *"Professor, os nossos testes não ficam parados apenas na máquina local. Nós automatizamos a esteira no GitHub. Sempre que alguém da equipe envia um código novo (um 'push'), o próprio servidor do GitHub Actions cria uma máquina virtual, instala o Python e roda esse mesmo comando da linha 29. Se alguém criar um código que quebra as regras da Pessoa 2, o GitHub acusa o erro com um 'X' vermelho e bloqueia a integração. Isso garante 100% de estabilidade."*

Como Executar o Sistema (Na Hora da Defesa)

Para garantir que tudo funcione na hora, siga estes passos exatos sem depender do docker:

1. **Abra o terminal do VSCode** (raiz do projeto) e crie o ambiente se não tiver: `python -m venv venv`
2. **Ative o ambiente:** (Windows) `.\venv\Scripts\Activate.ps1` ou (Linux/Mac) `source venv/bin/activate`
3. **Instale os requisitos:** `pip install -r requirements.txt`
4. **Instale os navegadores do Playwright:** `playwright install chromium`
5. **Prepare as variáveis (.env):**

- Copie o `.env.example` para um arquivo `.env`.
- Altere a linha do URL para: `GERP_URL=http://localhost:8000/web/gerp_fake.html`
- **IMPORTANTE:** Apague (ou comente com #) as linhas `DATA_DIR=/app/data`, `OUTPUT_DIR=/app/output` e `LOGS_DIR=/app/logs`. Como você está rodando fora do Docker, essas pastas `/app/` farão o sistema não achar os arquivos. Ao apagar, o código usará as pastas normais do projeto.

6. **Suba o servidor Mock:** Abra um **segundo terminal** (mantenha o primeiro lá). Entre na pasta: `cd gerp_fake_server`. Rode: `python -m http.server 8000`.
7. **Rode o Robô:** Volte no **primeiro terminal** (na raiz) e rode: `python -m src.main`.
8. **Rode os Testes (Sua hora de brilhar):** No mesmo terminal, rode `pytest -v` para provar ao professor que a qualidade e as regras de negócio estão garantidas.

5. Apresentando o GHCR (A Mágica da Nuvem - Slide 10)

O professor pede especificamente na Etapa 5 e 6 do documento para demonstrar o `docker pull` da imagem publicada no GHCR.

- **Onde mostrar:** Abra o seu repositório no GitHub pelo navegador. Olhe exatamente aí no **canto direito da tela**, onde está escrito **Packages**. Clique onde diz **atividade_avaliativa3** e mostre que a imagem está pública e hospedada lá.
- **A Conclusão (O que falar):** *"Como configuramos nossa pipeline com permissão de escrita (`packages: write`), nosso GitHub Actions não apenas testa o código, mas realiza o build da imagem Docker e automaticamente faz o push para o GitHub Container Registry. O artefato já está pronto para ir para produção."*
- **No Terminal (A cartada final):** Abra o terminal e digite ao vivo o comando para provar que a imagem desce da nuvem:

```
docker pull ghcr.io/codemartell/atividade_avaliativa3:latest
```

(Opcional: logo depois do pull, você pode rodar `docker run ghcr.io/codemartell/atividade_avaliativa3:latest` para mostrar o robô rodando direto da nuvem).